

Group_By_II

Isabela Diniz Cia

Instruções

- No arquivo purchase.sqlite encontram-se as tabelas PURCHASE criada especialmente para esta aula.
- Configure o Rstudio (Posit Cloud) para trabalhar com SQLite e importa o banco de dados purchase.sqlite

Configurando Rstudio

```
library(DBI)
library(RSQLite)
knitr::opts_chunk$set(max.print = Inf)

# Conecta/Cria arquivo
con <- dbConnect(RSQLite::SQLite(), "purchase.sqlite")
```

Tabela

```
SELECT * FROM PURCHASE
```

Table 1: 46 records

PNO	SNO	ENO	PJNO	PURDAY	COST	DISCOUNT
P1	S1	E1	PJ1	1	200	0
P1	S1	E1	PJ1	17	250	50

PNO	SNO	ENO	PJNO	PURDAY	COST	DISCOUNT
P1	S1	E1	PJ1	80	300	0
P1	S1	E1	PJ3	5	200	0
P1	S1	E1	PJ3	6	200	0
P1	S2	E2	PJ3	85	300	100
P2	S1	E1	PJ2	1	1000	0
P2	S1	E2	PJ2	2	1000	0
P2	S1	E2	PJ2	85	1200	0
P3	S1	E1	PJ1	10	500	100
P3	S1	E1	PJ1	11	500	100
P3	S1	E1	PJ1	70	550	200
P4	S2	E2	PJ2	10	300	0
P4	S1	E2	PJ2	11	300	0
P4	S2	E2	PJ2	70	400	50
P5	S1	E1	PJ1	11	1200	0
P5	S1	E1	PJ1	22	1200	0
P5	S1	E1	PJ1	33	1500	25
P6	S2	E2	PJ2	11	200	0
P6	S2	E2	PJ2	33	100	0
P7	S3	E1	PJ1	11	1000	0
P7	S4	E1	PJ1	22	1000	0
P7	S3	E1	PJ1	33	1000	0
P7	S4	E2	PJ2	61	1200	0
P7	S4	E2	PJ2	72	1200	200
P8	S1	E1	PJ1	61	100	0
P8	S1	E1	PJ1	72	100	0
P8	S1	E1	PJ1	73	100	0
P8	S1	E2	PJ2	41	100	50
P8	S2	E1	PJ2	42	100	75
P8	S1	E1	PJ2	43	100	100
P4	S1	E1	PJ3	11	300	75
P4	S1	E1	PJ3	22	300	75
P4	S2	E2	PJ3	33	300	0
P3	S3	E3	PJ1	7	1200	1100
P4	S3	E3	PJ1	8	1200	900
P5	S3	E3	PJ1	9	1500	1400
P3	S3	E3	PJ2	14	1200	1000
P4	S3	E3	PJ2	15	1200	1000
P5	S3	E3	PJ2	16	1500	1300
P3	S4	E4	PJ1	10	1100	0
P4	S4	E4	PJ1	15	1200	0
P5	S4	E4	PJ2	20	1500	50

PNO	SNO	ENO	PJNO	PURDAY	COST	DISCOUNT
P3	S4	E4	PJ2	21	1100	50
P4	S4	E4	PJ1	70	1200	0
P6	S4	E4	PJ2	71	1500	0

Cada linha da tabela **PURCHASE** contém informações sobre a compra de uma peça de um fornecedor, realizada por um funcionário, para uso em um projeto

- Cada peça é identificada por um número da peça (**PNO**);
- Cada fornecedor é identificado por um número do fornecedor (**SNO**);
- Cada funcionário é identificado por um número do funcionário (**ENO**);
- Cada projeto é identificado por um número do projeto (**PJNO**).

Essas quatro colunas contêm dados do tipo cadeia de caracteres (string). As próximas três colunas contêm valores inteiros. São elas:

- **PURDAY**: Data da compra. Aqui, 1 representa o primeiro dia do ano corrente, 2 representa o segundo dia do ano corrente, e assim por diante.
- **COST**: Custo da peça, em dólares.
- **DISCOUNT**: Valor do desconto, que pode variar de zero até o valor total de **COST** da peça.

GROUP BY

1. Consulte a tabela **PURCHASE**. Para cada projeto, exiba seu número de projeto (**PJNO**), seguido pelo custo total de todas as peças compradas para o projeto. Ordene o resultado pelos valores de **PJNO**.

```
SELECT PJNO, SUM(COST) AS CUSTO_TOTAL
FROM PURCHASE
GROUP BY PJNO
ORDER BY PJNO
```

Table 2: 3 records

PJNO	CUSTO_TOTAL
PJ1	16900
PJ2	15200

PJNO	CUSTO_TOTAL
PJ3	1600

2. Consulte a tabela **PURCHASE**. Para cada fornecedor, exiba seu número de fornecedor (**SNO**), seguido pelo custo total de todas as peças compradas do fornecedor.

```
SELECT SNO, SUM(COST) AS CUSTO_TOTAL_PECAS
FROM PURCHASE
GROUP BY SNO
```

Table 3: 4 records

SNO	CUSTO_TOTAL_PECAS
S1	11200
S2	1700
S3	9800
S4	11000

3. Consulte a tabela **PURCHASE**. Para cada projeto que utilizou peças fornecidas por algum fornecedor, exiba o número do projeto (**PJNO**), seguido pelo número do fornecedor (**SNO**) e pelo custo total das peças do projeto compradas desse fornecedor. Ordene o resultado por **SNO** dentro de cada **PJNO**.

```
SELECT PJNO, SNO, SUM(COST) AS SOMA_CUSTO
FROM PURCHASE
GROUP BY PJNO, SNO
ORDER BY PJNO, SNO
```

Table 4: 9 records

PJNO	SNO	SOMA_CUSTO
PJ1	S1	6500
PJ1	S3	5900
PJ1	S4	4500
PJ2	S1	3700
PJ2	S2	1100
PJ2	S3	3900
PJ2	S4	6500
PJ3	S1	1000

PJNO	SNO	SOMA_CUSTO
PJ3	S2	600

4. Para cada fornecedor que vendeu pelo menos uma peça para ser utilizada em algum projeto, exiba o valor **SNO** do fornecedor, seguido pelo valor **PJNO** do projeto e pelo custo total das peças que o fornecedor forneceu para o projeto. Ordene o resultado por **PJNO** dentro de cada **SNO**.

```
SELECT SNO, PJNO, SUM(COST)
FROM PURCHASE
GROUP BY SNO, PJNO
ORDER BY SNO, PJNO
```

Table 5: 9 records

SNO	PJNO	SUM(COST)
S1	PJ1	6500
S1	PJ2	3700
S1	PJ3	1000
S2	PJ2	1100
S2	PJ3	600
S3	PJ1	5900
S3	PJ2	3900
S4	PJ1	4500
S4	PJ2	6500

5. Consulte a tabela **PURCHASE**. Para cada funcionário que comprou peças de um fornecedor, exiba o valor **ENO** do funcionário, seguido pelo valor **SNO** do fornecedor e, em seguida, pelo custo total das peças que o funcionário comprou desse fornecedor.

```
SELECT ENO, SNO, SUM(COST) AS CUSTO
FROM PURCHASE
GROUP BY ENO, SNO
```

Table 6: 9 records

ENO	SNO	CUSTO
E1	S1	8600
E1	S2	100

ENO	SNO	CUSTO
E1	S3	2000
E1	S4	1000
E2	S1	2600
E2	S2	1600
E2	S4	2400
E3	S3	7800
E4	S4	7600

6. Consulte a tabela `PURCHASE`. Para cada projeto em que um funcionário comprou uma peça de um fornecedor para ser utilizada no projeto, exiba os respectivos valores `PJNO`, `ENO` e `SNO`, seguidos pelo custo total das peças compradas pelo funcionário do fornecedor para o projeto. Ordene o resultado por `SNO` dentro de `ENO`, dentro de `PJNO`.

```
SELECT PJNO, ENO, SNO, SUM(COST) AS CUSTO
FROM PURCHASE
GROUP BY PJNO, ENO, SNO
ORDER BY PJNO, ENO, SNO
```

Table 7: 14 records

PJNO	ENO	SNO	CUSTO
PJ1	E1	S1	6500
PJ1	E1	S3	2000
PJ1	E1	S4	1000
PJ1	E3	S3	3900
PJ1	E4	S4	3500
PJ2	E1	S1	1100
PJ2	E1	S2	100
PJ2	E2	S1	2600
PJ2	E2	S2	1000
PJ2	E2	S4	2400
PJ2	E3	S3	3900
PJ2	E4	S4	4100
PJ3	E1	S1	1000
PJ3	E2	S2	600

7. Consulte a tabela `PURCHASE`. Para cada projeto em que um funcionário comprou uma peça de um fornecedor para ser utilizada no projeto, exiba os respectivos valores `PJNO`, `ENO`, `SNO` e `PNO`, seguidos pelo custo total de cada peça comprada pelo funcionário do

fornecedor para o projeto. Ordene o resultado por PNO dentro de SNO, dentro de ENO, dentro de PJNO.

```
SELECT PJNO, ENO, SNO, PNO, SUM(COST)
FROM PURCHASE
GROUP BY PJNO, ENO, SNO, PNO
ORDER BY PJNO, ENO, SNO, PNO
```

Table 8: 30 records

PJNO	ENO	SNO	PNO	SUM(COST)
PJ1	E1	S1	P1	750
PJ1	E1	S1	P3	1550
PJ1	E1	S1	P5	3900
PJ1	E1	S1	P8	300
PJ1	E1	S3	P7	2000
PJ1	E1	S4	P7	1000
PJ1	E3	S3	P3	1200
PJ1	E3	S3	P4	1200
PJ1	E3	S3	P5	1500
PJ1	E4	S4	P3	1100
PJ1	E4	S4	P4	2400
PJ2	E1	S1	P2	1000
PJ2	E1	S1	P8	100
PJ2	E1	S2	P8	100
PJ2	E2	S1	P2	2200
PJ2	E2	S1	P4	300
PJ2	E2	S1	P8	100
PJ2	E2	S2	P4	700
PJ2	E2	S2	P6	300
PJ2	E2	S4	P7	2400
PJ2	E3	S3	P3	1200
PJ2	E3	S3	P4	1200
PJ2	E3	S3	P5	1500
PJ2	E4	S4	P3	1100
PJ2	E4	S4	P5	1500
PJ2	E4	S4	P6	1500
PJ3	E1	S1	P1	400
PJ3	E1	S1	P4	600
PJ3	E2	S2	P1	300
PJ3	E2	S2	P4	300

8. Consulte a tabela **PURCHASE**. Considere apenas as compras realizadas pelo funcionário **E1**. Para cada fornecedor do qual o funcionário **E1** comprou uma peça, exiba os respectivos valores **ENO** e **SNO**, seguidos pelo custo total, custo máximo e custo mínimo das compras realizadas pelo funcionário **E1** de cada fornecedor.

```
SELECT SNO, ENO, SUM(COST) AS CUSTO, MAX(COST) AS MAXIMO, MIN(COST) AS MINIMO
FROM PURCHASE
WHERE ENO = "E1"
GROUP BY ENO, SNO
```

Table 9: 4 records

SNO	ENO	CUSTO	MAXIMO	MINIMO
S1	E1	8600	1500	100
S2	E1	100	100	100
S3	E1	2000	1000	1000
S4	E1	1000	1000	1000

9. Estamos interessados em saber se algum fornecedor vendeu a mesma peça por preços diferentes (ou seja, se todas as compras que apresentam a mesma combinação de valores (**SNO**, **PNO**) possuem o mesmo valor de **COST**). Para qualquer fornecedor que tenha cobrado preços diferentes pela mesma peça, exiba as seguintes informações:
- Os respectivos valores de **SNO** e **PNO**;
 - O **COST** mínimo (**MINCOST**) para cada par de valores (**SNO**, **PNO**);
 - O **COST** máximo (**MAXCOST**) para cada par de valores (**SNO**, **PNO**);
 - A diferença (**COSTDIFF**) entre os custos máximo e mínimo;
 - Exiba apenas as linhas em que os custos máximo e mínimo sejam diferentes;
 - Exiba as informações na seguinte sequência de colunas: **SNO**, **PNO**, **MAXCOST**, **MINCOST**, **COSTDIFF**;
 - Ordene o resultado em ordem decrescente por **COSTDIFF**. Em caso de valores de **COSTDIFF** iguais, ordene por **PNO** (crescente) dentro de **SNO** (crescente).

```
SELECT SNO, PNO,
       MAX(COST) AS MAXCOST,
       MIN(COST) AS MINCOST,
       (MAX(COST) - MIN(COST)) AS COSTDIFF
FROM PURCHASE
```

```
GROUP BY SNO, PNO
HAVING COSTDIFF <> 0
ORDER BY COSTDIFF DESC, SNO ASC, PNO ASC
```

Table 10: 7 records

SNO	PNO	MAXCOST	MINCOST	COSTDIFF
S1	P5	1500	1200	300
S1	P2	1200	1000	200
S4	P7	1200	1000	200
S1	P1	300	200	100
S2	P4	400	300	100
S2	P6	200	100	100
S1	P3	550	500	50

FUNÇÕES INDIVIDUAIS

Definição

Uma função individual opera sobre um valor de uma única linha e retorna um único valor para essa mesma linha.

- ROUND(N, M) — Arredonda N para M casas decimais.
- TRUNC(N, M) — Trunca N para M casas decimais. [SQLite: não funciona (mas pode tentar com CAST())]
- ABS(N) — Retorna o valor absoluto de N.
- SIGN(N) — Retorna 1 se N > 0, 0 se N = 0 e -1 se N < 0.
- MOD(N, M) — Retorna o resto da divisão de N por M. [SQLite: %]
- POWER(N, M) — Retorna N elevado à potência M.
- SQRT(N) — Retorna a raiz quadrada de N.

```
dbDisconnect(con)
knitr::opts_chunk$set(max.print = Inf)
# Conecta/Cria o arquivo
con <- dbConnect(RSQLite::SQLite(), "demo2.sqlite")
```

```
SELECT * FROM DEMO2
```

Table 11: 5 records

I1	D1	V1	F1
-10	-8.82	Julie Martyn	Hello
-5	-5.28	JESSIE MARTYN	GOOD
0	0.00	Janet Martyn	By
2	6.42	Frank	BYE
9	9.98	Wally	HYY

10. Aplicar na coluna D1 da tabela DEMO2 todas as funções individuais listadas.

```
SELECT D1,
ROUND(D1, 1) AS ROUND1,
CAST(D1 AS INTEGER) AS TRUNCO,
ABS(D1) AS ABSVAL,
SIGN(D1) AS SINAL,
ROUND(D1, 0) % 2 AS RESTO,
POWER(D1, 2) AS QUADRADO,
POWER(D1, 3) AS CUBO,
SQRT(ABS(D1)) AS RAIZ
FROM DEMO2
```

Table 12: 5 records

D1	ROUND1	TRUNC0	AB- SVAL	SINAL	RESTO	QUADRADO	CUBO	RAIZ
-8.82	-8.8	-8	8.82	-1	-1	77.7924	-686.1290	2.969848
-5.28	-5.3	-5	5.28	-1	-1	27.8784	-147.1980	2.297825
0.00	0.0	0	0.00	0	0	0.0000	0.0000	0.000000
6.42	6.4	6	6.42	1	0	41.2164	264.6093	2.533772
9.98	10.0	9	9.98	1	0	99.6004	994.0120	3.159114

11. Calcule a soma dos valores absolutos de D1.

```
SELECT SUM(ABS(D1)) FROM DEMO2
```

Table 13: 1 records

SUM(ABS(D1))
30.5

12. Também existem funções individuais que podemos aplicar a **STRINGS**. Veremos elas no exemplo a seguir:

```
SELECT V1, F1,
LENGTH(V1),
UPPER(F1),
LOWER(V1),
LTRIM(V1),
RTRIM(V1),
SUBSTR(V1, 3, 5),
CONCAT(V1, UPPER(F1))
FROM DEMO2
```

Table 14: 5 records

V1	F1	LENGTH(V1)	UPPER(F1)	LOWER(V1)	LTRIM(V1)	RTRIM(V1)	SUBSTR(V1, 3, 5)	CONCAT(V1, UPPER(F1))
Julie Martyn	Hello	12	HELLO	julie martyn	Julie Martyn	Julie Martyn	lie M	Julie Martyn-HELLO
JESSIE MAR-TYN	GOOD	13	GOOD	jessie martyn	JESSIE MAR-TYN	JESSIE MAR-TYN	SSIE	JESSIE MARTYN-GOOD
Janet Martyn	By	12	BY	janet martyn	Janet Martyn	Janet Martyn	net M	Janet MartynBY
Frank Wally	BYE HYY	5 5	BYE HYY	frank wally	Frank Wally	Frank Wally	ank lly	FrankBYE WallyHYY

Funções individuais para STRING

Função operador -> Descrição
LENGTH(C) -> Retorna o comprimento de C.
UPPER(C) -> Retorna C em letras maiúsculas.
LOWER(C) -> Retorna C em letras minúsculas.
LTRIM(C) -> Retorna C removendo os espaços em branco à esquerda.
RTRIM(C) -> Retorna C removendo os espaços em branco à direita.
SUBSTR(C, I, L) -> Retorna uma substring de C, iniciando no caractere I e com comprimento L.
CONCAT(C1, C2) -> Concatena os valores de C1 e C2.

13. Qual a diferença entre TRIM() e REPLACE()?

```
SELECT V1, TRIM(V1), REPLACE(V1, ' ', '') FROM DEMO2
```

Table 15: 5 records

V1	TRIM(V1)	REPLACE(V1, ' ', '')
Julie Martyn	Julie Martyn	JulieMartyn
JESSIE MARTYN	JESSIE MARTYN	JESSIEMARTYN
Janet Martyn	Janet Martyn	JanetMartyn
Frank	Frank	Frank
Wally	Wally	Wally